



UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

INGENIERÍA DE SOFTWARE

Análisis y Diseño de Software
30724

Informe Plan de Pruebas **Sistema EVENTUAL**

Presenta:

Cuzco Yamasca Alex Darío

Domínguez Aguilera Pablo Alejandro

Alvarado Palacios Dylan Alexander

Docente:

Ing. Jenny Ruiz

Fecha:

9 de Julio de 2026



Índice

1. Objetivos	3
a. Objetivo General	3
b. Objetivos Específicos	3
2. Planteamiento	4
a. Alcance de las Pruebas	4
b. Enfoque de Testing	4
c. Estrategia de Mocking	5
3. Herramienta	6
a. Framework de Testing	6
b. Entorno de Pruebas	6
c. Estructura de archivos	6
4. Implementación	7
a. RF01 - Acceso al Software	7
b. RF02 - Gestión de Miembros	10
c. RF03 - Consultar Calendario de Eventos	11
d. RF04 - Proponer Evento	12
e. RF05 - Confirmar Asistencia a Evento	13
f. RF06 - Registrar Gastos del Evento	15
g. RF07 - Registrar Aportes Económicos	16
h. RF08 - Registrar Evento	16
i. RF09 - Difundir Información del Evento	18
j. RF10 - Registrar Proveedores	19
k. RF11 - Definir Evento	20
l. RF12 - Ejecutar Evento	21
m. RF13 - Generar Reportes	22
5. Resultados	24
6. Problemas y soluciones	27
7. Conclusiones	32
8. Recomendaciones	34
9. Anexos	35

Informe Plan de Pruebas

Proyecto: EVENTUAL v.1.0 - Sistema de Gestión de Eventos Sociales

Fecha de Creación: 7 de Julio 2026

Fecha de Ejecución: 8 de Julio 2026

Versión: 1.0

Estado: Ejecutado - 100% Aprobado

Estado de Implementación

Métrica	Valor
Tests Planificados	137
Tests Ejecutados	137
Tiempo de Ejecución Real	~6s (backend) + ~3s (frontend)
Suites de Tests	26/26
Requisitos Implementados	13/13 (100%)
Tasa de Éxito Alcanzada	100%

Requisitos Funcionales - Estado Final

RF	Nombre	Tests	Tasa Éxito
RF01	Acceso al Software	12	100%
RF02	Gestión de Miembros	16	100%
RF03	Consultar Calendario de Eventos	11	100%
RF04	Proponer Evento	10	100%
RF05	Confirmar Asistencia a Evento	11	100%
RF06	Registrar Gastos del Evento	10	100%
RF07	Registrar Aportes Económicos	10	100%

RF08	Registrar Evento	9	100%
RF09	Difundir Información del Evento	8	100%
RF10	Registrar Proveedores	5	100%
RF11	Definir Evento	13	100%
RF12	Ejecutar Evento	11	100%
RF13	Generar Reportes	11	100%
Total		137	100%

Plan de Pruebas

1. Objetivos

a. Objetivo General

Validar el cumplimiento de los requisitos funcionales del sistema EVENTUAL v.1.0 mediante la ejecución de pruebas unitarias sobre los componentes del backend y frontend, con el fin de verificar el correcto funcionamiento de sus funcionalidades, reglas de negocio y manejo de errores.

b. Objetivos Específicos

- Validar el funcionamiento de los controladores del backend para verificar que los servicios respondan correctamente ante escenarios válidos, inválidos y de borde.
- Validar las reglas de negocio implementadas para comprobar que las restricciones y validaciones del sistema se cumplan correctamente en cada proceso.
- Validar el manejo de errores de la aplicación para verificar que las excepciones y condiciones de fallo sean gestionadas de manera adecuada.

2. Planteamiento

a. Alcance de las Pruebas

Las pruebas unitarias cubren:

- Backend
 - Controladores (13 archivos): Pruebas HTTP con Supertest sobre cada ruta Express, mockeando el cliente Supabase.
 - Middlewares: Autenticación JWT, bloqueo por intentos, validación de roles.
- Frontend
 - BLoCs (12 archivos): Pruebas de estados con bloc_test, mockeando ApiClient o repositorios abstractos.

Fuera del alcance:

- Pruebas de integración real contra Supabase.
- Pruebas E2E con navegador/dispositivo.
- Pruebas de carga o rendimiento.
- Pruebas de seguridad (penetration testing).

b. Enfoque de Testing

Metodología: TDD adaptado (los tests se diseñan contra la implementación existente, documentando el comportamiento esperado)

Tipo de Pruebas: Unitarias

Cobertura Objetivo: 80% mínimo de cobertura de código (Meta: 100% de tests aprobados).

Resumen:

- E2E: fuera de alcance - fase futura.
- Integración: futuro: tests con Supabase real.
- Unitarias: 137 tests (100% de la estrategia actual).

c. Estrategia de Mocking

Backend (Jest + Supertest):

- Supabase client se mockea a nivel de módulo con jest.mock() para cada controlador
- Cada mock retorna datos controlados por test (exit, error, empty, etc.)
- Las respuestas HTTP se validan con supertest (status + body)
- Los tokens JWT se generan con jsonwebtoken.sign() en los tests que requieren autenticación

```
// Patrón de mock para Supabase
jest.mock('../src/config/supabase', () => ({
  supabase: {
    from: jest.fn().mockReturnThis(),
    select: jest.fn().mockReturnThis(),
    eq: jest.fn().mockReturnThis(),
    single: jest.fn(),
    insert: jest.fn(),
    update: jest.fn(),
    delete: jest.fn(),
  },
}));
```

Frontend (mocktail + bloc_test):

- ApiClient se mockea con Mocktail para 10 de 13 BLoCs que dependen directamente de él
- AuthRepository y MembersRepository se mockean a través de sus interfaces abstractas (3 BLoCs)
- Los mocks se registran con when(() => mock.method(params)).thenAnswer(...) y se inyectan vía get_it
- blocTest proporciona el scaffolding de Arrange → Act → Assert

```
// Patrón de mock para BLoC con mocktail
class MockApiClient extends Mock implements ApiClient {}

void main() {
  late EventsBloc bloc;
  late MockApiClient mockApiClient;

  setUp(() {
    mockApiClient = MockApiClient();
    bloc = EventsBloc(apiClient: mockApiClient);
  });
}
```

3. Herramienta

a. Framework de Testing

i. Backend

Herramienta	Versión	Propósito
Jest	^29.7.0	Framework de testing
Supertest	^7.0.0	Pruebas HTTP sobre Express

ii. Frontend

Herramienta	Versión	Propósito
flutter_test	(SDK)	Framework base de Flutter
bloc_test	^9.1.6	Pruebas especializadas para BLoC
mocktail	^1.0.4	Mocking sin code generation

b. Entorno de Pruebas

- Backend: Node.js v18+, npm test via npx jest --coverage
- Frontend: Flutter SDK 3.x, flutter test --coverage
- Sistema Operativo: Windows 11
- Base de datos: Mockeada (no se requiere Supabase en ejecución).

c. Estructura de archivos

```
backend/  
  __tests__/  
    controllers/  
      auth.test.js           (6 tests)  
      members.test.js       (10 tests)  
      events.test.js        (5 tests)  
      proposals.test.js     (5 tests)  
      attendance.test.js    (6 tests)  
      expenses.test.js      (5 tests)  
      contributions.test.js (5 tests)  
      eventRegistration.test.js (4 tests)  
      broadcast.test.js     (3 tests)  
      providers.test.js     (5 tests)
```

```

planEvent.test.js          (5 tests)
quotations.test.js        (3 tests)
executeEvent.test.js       (5 tests)
reports.test.js           (5 tests)

frontend/
test/
  bloc/
    auth_bloc_test.dart    (6 tests)
    members_bloc_test.dart (6 tests)
    events_bloc_test.dart  (6 tests)
    proposals_bloc_test.dart (5 tests)
    attendance_bloc_test.dart (5 tests)
    expenses_bloc_test.dart (5 tests)
    contributions_bloc_test.dart (5 tests)
    event_registration_bloc_test.dart (5 tests)
    broadcast_bloc_test.dart (5 tests)
    plan_event_bloc_test.dart (5 tests)
    execute_event_bloc_test.dart (6 tests)
    reports_bloc_test.dart (6 tests)
  helpers/
    test_setup.dart

```

Total: 26 archivos de prueba (14 backend + 12 frontend)

4. Implementación

a. RF01 - Acceso al Software

Archivos:

- backend/___tests___/controllers/auth.test.js (6 tests)
- frontend/test/bloc/auth_bloc_test.dart (6 tests)

Total de Tests: 12 pruebas — Estado: APROBADO

Cómo se Implementó

Backend - Controlador de autenticación:

Se probaron los siguientes escenarios contra POST /api/auth/login y el middleware JWT:

1. Login exitoso: cédula de 10 dígitos con contraseña correcta → 200 + token JWT + datos del socio.
2. Cédula no registrada: 401 + mensaje con conteo de intentos.
3. Contraseña incorrecta (3 intentos): 423 + "Cuenta bloqueada por seguridad".
4. Cuenta inactiva: 403 + "Su cuenta está inactiva".
5. Token JWT expirado en rutas protegidas: 401 + "Token inválido o expirado".

6. Cédula con formato inválido: 400 + error de validación.

```
describe('RF01.1 - Login exitoso', () => {
  test('Debe retornar 200 con token JWT', async () => {
    const socioMock = {
      id_socio: 1,
      cedula: '1234567890',
      password_hash: await bcrypt.hash('password123', 10),
      estado: 'Activo',
      id_rol: 3,
      nombre_completo: 'Juan Pérez',
    };

    supabase.from().select().eq().single.mockResolvedValue({
      data: socioMock, error: null });

    const res = await request(app)
      .post('/api/auth/login')
      .send({ cedula: '1234567890', password: 'password123' });

    expect(res.status).toBe(200);
    expect(res.body.token).toBeDefined();
    expect(res.body.user.nombre_completo).toBe('Juan Pérez');
  });
});
```

```
describe('RF01.3 - Bloqueo por 3 intentos', () => {
  test('Debe retornar 423 tras 3 intentos fallidos', async () => {
    {
      // Arrange: mock de socio activo
      const socioMock = {
        id_socio: 1, cedula: '1234567890',
        password_hash: await bcrypt.hash('real', 10),
        estado: 'Activo',
        login_attempts: 2, locked_until: null,
      };
      supabase.from().select().eq().single.mockResolvedValue({
        data: socioMock, error: null });
      supabase.from().update().eq().mockResolvedValue({ error: null
    });

    // Act: tercer intento fallido
    const res = await request(app)
      .post('/api/auth/login')
      .send({ cedula: '1234567890', password: 'wrong' });

    // Assert
    expect(res.status).toBe(423);
    expect(res.body.message).toContain('Cuenta bloqueada');
  });
});
```

Frontend — Auth BLoC:

```

void main() {
  late AuthBloc bloc;
  late MockAuthRepository mockRepo;

  setUp(() {
    mockRepo = MockAuthRepository();
    bloc = AuthBloc(authRepository: mockRepo);
  });

  blocTest<AuthBloc, AuthState>(
    'RF01.1 - Login exitoso → AuthAuthenticated',
    build: () => bloc,
    act: (bloc) => bloc.add(AuthLoginRequested(
      cedula: '1234567890', password: 'password123',
    )),
    setUp: () {
      when(() => mockRepo.login(
        cedula: any(named: 'cedula'),
        password: any(named: 'password'),
      )).thenAnswer((_) async => AuthResult(
        user: UserEntity(id: 1, nombreCompleto: 'Juan Pérez'),
        failure: null,
      ));
    },
    expect: () => [isA<AuthAuthenticated>()],
  );

  blocTest<AuthBloc, AuthState>(
    'RF01.3 - Cuenta bloqueada → AuthBlocked',
    build: () => bloc,
    act: (bloc) => bloc.add(AuthLoginRequested(
      cedula: '1234567890', password: 'wrong',
    )),
    setUp: () {
      when(() => mockRepo.login(
        cedula: any(named: 'cedula'),
        password: any(named: 'password'),
      )).thenAnswer((_) async => AuthResult(
        user: null,
        failure: AuthFailure('Cuenta bloqueada por seguridad. Intente en
15 minutos.'),
      ));
    },
    expect: () => [isA<AuthBlocked>()],
  );
}

```

Resultados Obtenidos

Funcionalidad de autenticación validada:

- Login exitoso retorna token JWT con datos del socio.
- Conteo de intentos fallidos hasta 3.
- Bloqueo de cuenta por 15 minutos.
- Validación de formato de cédula.

- Manejo de cuentas inactivas.
- Protección de rutas con middleware JWT.
- Manejo de logout y limpieza de sesión.

b. RF02 - Gestión de Miembros

Archivos:

- backend/___tests___/controllers/members.test.js (10 tests)
- frontend/test/bloc/members_bloc_test.dart (6 tests)

Total de Tests: 16 pruebas — Estado: APROBADO

Cómo se Implementó

Backend — Controlador de miembros:

Se probaron los endpoints CRUD protegidos por rol Presidente:

1. Listar miembros: GET /api/members → 200 + array con rol
2. Lista vacía: GET /api/members → 200 + { members: [] }
3. Crear socio válido: POST /api/members → 201
4. Email duplicado: POST /api/members → 409
5. Cédula duplicada: POST /api/members → 409
6. Actualizar socio: PUT /api/members/:id → 200
7. Socio no encontrado: PUT /api/members/:id → 404
8. Asignar rol: PATCH /api/members/:id/role → 200
9. Desactivar socio: PATCH /api/members/:id/deactivate → 200
10. Auto-desactivación: mismo ID del token → 400
11. Acceso sin rol Presidente: 403

```
describe('RF02.3 - Crear socio exitosamente', () => {
  test('Debe retornar 201 con datos completos', async () => {
    const datosValidos = {
      cedula: '0987654321',
      nombre_completo: 'María García',
      email: 'maria@example.com',
      telefono: '0999999999',
      direccion: 'Av. Siempre Viva',
    };

    supabase.from().insert().select().single
      .mockResolvedValue({ data: { id_socio: 2, ...datosValidos }, error: null });

    const res = await request(app)
      .post('/api/members')
```

```

        .set('Authorization', `Bearer ${presidenteToken}`)
        .send(datosValidos);

        expect(res.status).toBe(201);
        expect(res.body.message).toBe('Socio creado exitosamente');
    });
});

```

Resultados Obtenidos

- CRUD completo de socios con roles.
- Prevención de duplicados (email y cédula).
- Validación de permisos por rol.
- Protección contra auto-desactivación.
- Manejo de socio no encontrado.

c. RF03 - Consultar Calendario de Eventos

Archivos:

- backend/___tests___/controllers/events.test.js (5 tests)
- frontend/test/bloc/events_bloc_test.dart (6 tests)

Total de Tests: 11 pruebas — Estado: APROBADO

Cómo se Implementó

Backend — Controlador de eventos públicos:

1. Listar eventos sin filtros → GET /api/events → 200 + todos los eventos activos
2. Filtrar por tipo → GET /api/events?tipo=Social → 200 + filtrados
3. Filtrar por mes/año → GET /api/events?year=2026&month=8 → 200
4. Sin eventos → GET /api/events?year=2099 → 200 + []
5. Detalle de evento → GET /api/events/:id → 200
6. Evento no encontrado → 404
7. Propuesta no visible (estado Propuesta no se lista)

```

describe('RF03.1 - Listar eventos sin filtros', () => {
  test('Debe retornar 200 con array de eventos', async () => {
    supabase.from().select().in().mockResolvedValue({
      data: [{ id_evento: 1, nombre: 'Fiesta', tipo_evento:
'Social' }],
      error: null,
    });
  });
});

```

```
const res = await request(app).get('/api/events');

expect(res.status).toBe(200);
expect(Array.isArray(res.body.eventos)).toBe(true);
});
});
```

Resultados Obtenidos

- Listado público de eventos (solo estados visibles).
- Filtros por tipo, mes y año.
- Manejo de listas vacías.
- Detalle individual de evento.

d. RF04 - Proponer Evento

Archivos:

- backend/___tests___/controllers/proposals.test.js (7 tests)
- frontend/test/features/proposals/bloc/proposals_bloc_test.dart (4 tests)

Total de Tests: 10 pruebas — Estado: APROBADO

Cómo se Implementó

Backend — Controlador de propuestas:

1. Propuesta creada exitosamente: POST /api/proposals → 201 + número de seguimiento PROP-...
2. Descripción menor a 50 caracteres: 400
3. Tipo de evento inválido (no Social/Deportivo/Familiar): 400
4. Socio no autenticado: 401
5. Obtener mis propuestas: GET /api/proposals/mine → 200
6. Sin propuestas: 200 + []
7. Error de base de datos: 500

```
describe('RF04.1 - Propuesta creada exitosamente', () => {
  test('Debe retornar 201 con código de seguimiento', async () => {
    {
      supabase.from().insert().select().single()
      .mockResolvedValue({
        data: { id_propuesta: 1, codigo_seguimiento:
'PROP-2026-001' },

```

```

        error: null,
      });

      const res = await request(app)
        .post('/api/proposals')
        .set('Authorization', `Bearer ${socioToken}`)
        .send({
          tipo_evento: 'Social',
          descripcion: 'X'.repeat(51),
          fecha_sugerida: '2026-12-25',
          justificacion: 'Fiesta de fin de año',
        });

      expect(res.status).toBe(201);

      expect(res.body.propuesta.codigo_seguimiento).toMatch(/^PROP-/);
    });
  });
});

```

Resultados Obtenidos

- Creación de propuestas con código de seguimiento automático.
- Validación de longitud mínima de descripción (50 caracteres).
- Validación de tipo de evento contra catálogo.
- Listado de propuestas por socio autenticado.

e. RF05 - Confirmar Asistencia a Evento

Archivos:

- backend/__tests__/controllers/attendance.test.js (6 tests)
- frontend/test/bloc/attendance_bloc_test.dart (5 tests)

Total de Tests: 11 pruebas — Estado: APROBADO

Cómo se Implementó

Backend — Controlador de asistencia:

1. Confirmar asistencia exitosamente: POST /api/events/:id/attendance → 201
2. Cero acompañantes: 201 válido
3. Evento no encontrado: 404
4. Evento en estado incorrecto (no Registrado/Difundido): 400
5. Menos de 15 días de anticipación: 400
6. Confirmación duplicada: 409
7. Cupo máximo excedido: 400

8. Socio no autenticado: 401
9. Obtener mi confirmación: GET /:eventoId/mine → 200

```
describe('RF05.1 - Confirmar asistencia exitosamente', () => {
  test('Debe retornar 201 con datos de confirmación', async () => {
    {
      supabase.from().select().eq().single()
        .mockResolvedValueOnce({ data: { id_evento: 1, estado:
        'Registrado', cupo_maximo: 100, fecha: '2026-12-25' }, error:
        null });

      supabase.from().select().eq().single()
        .mockResolvedValueOnce({ data: null, error: null });

      supabase.from().insert().select().single()
        .mockResolvedValue({ data: { id_confirmacion: 1 }, error:
        null });

      const res = await request(app)
        .post('/api/events/1/attendance')
        .set('Authorization', `Bearer ${socioToken}`)
        .send({ asiste: true, num_acompanantes: 2 });

      expect(res.status).toBe(201);
    });

    test('Debe retornar 409 si ya confirmó', async () => {
      supabase.from().select().eq().single()
        .mockResolvedValueOnce({ data: { id_evento: 1, estado:
        'Registrado', fecha: '2026-12-25' }, error: null });

      supabase.from().select().eq().single()
        .mockResolvedValueOnce({ data: { id_confirmacion: 5 },
        error: null });

      const res = await request(app)
        .post('/api/events/1/attendance')
        .set('Authorization', `Bearer ${socioToken}`)
        .send({ asiste: true, num_acompanantes: 0 });

      expect(res.status).toBe(409);
      expect(res.body.message).toContain('Ya ha confirmado');
    });
  });
});
```

Resultados Obtenidos

- Confirmación con acompañantes.
- Validación de ventana de 15 días antes del evento.
- Prevención de confirmaciones duplicadas.
- Control de cupo máximo.
- Validación de estado del evento.

f. RF06 - Registrar Gastos del Evento

Archivos:

- backend/___tests___/controllers/expenses.test.js (5 tests)
- frontend/test/bloc/expenses_bloc_test.dart (5 tests)

Total de Tests: 10 pruebas — Estado: APROBADO

Cómo se Implementó

Backend — Controlador de gastos:

1. Registrar gasto exitosamente: POST /api/events/:id/expenses → 201 (monto < saldo presupuesto)
2. Presupuesto excedido: 400 + "Presupuesto insuficiente"
3. Alerta al 90% del presupuesto: 201 + alert: true
4. Evento no encontrado: 404
5. Rol incorrecto (no Tesorero): 403
6. Listar gastos: GET /:eventoId → 200 + array
7. Evento sin gastos: 200 + []

```
describe('RF06.2 - Presupuesto excedido', () => {
  test('Debe retornar 400 si el gasto excede el presupuesto',
    async () => {
      supabase.from().select().eq().single()
        .mockResolvedValue({ data: { id_evento: 1,
          presupuesto_total: 100, total_gastos: 95 }, error: null });

      const res = await request(app)
        .post('/api/events/1/expenses')
        .set('Authorization', `Bearer ${tesoreroToken}`)
        .send({ concepto: 'Decoración', monto: 20, categoria:
          'Logística' });

      expect(res.status).toBe(400);
      expect(res.body.message).toContain('Presupuesto
        insuficiente');
    });
});
```

Resultados Obtenidos

- Control de presupuesto en tiempo real.
- Alerta automática al alcanzar el 90% del presupuesto.
- Validación de rol Tesorero.
- Listado histórico de gastos por evento.

g. RF07 - Registrar Aportes Económicos

Archivos:

- backend/___tests___/controllers/contributions.test.js (5 tests)
- frontend/test/bloc/contributions_bloc_test.dart (5 tests)

Total de Tests: 10 pruebas — Estado: APROBADO

Cómo se Implementó

Backend — Controlador de aportes:

1. Registrar aporte exitoso: POST /api/contributions → 201 (monto >= \$20)
2. Aporte duplicado en el mismo período: 409
3. Monto mínimo no cumplido: 400 + "El monto mínimo es \$20"
4. Listar socios pendientes: GET /api/contributions/pending → 200
5. Rol incorrecto (no Tesorero): 403
6. Todos al día: GET /pending → 200 + []

```
describe('RF07.3 - Monto mínimo no cumplido', () => {
  test('Debe retornar 400 para montos menores a $20', async () => {
    {
      const res = await request(app)
        .post('/api/contributions')
        .set('Authorization', `Bearer ${tesoreroToken}`)
        .send({ socio_id: 1, monto: 10, periodo: '2026-07' });

      expect(res.status).toBe(400);
      expect(res.body.message).toContain('monto mínimo');
    });
  });
});
```

Resultados Obtenidos

- Validación de monto mínimo (\$20).
- Prevención de aportes duplicados por período.
- Listado de morosos para gestión de cobro.
- Control de acceso por rol Tesorero.

h. RF08 - Registrar Evento

Archivos:

- backend/___tests___/controllers/eventRegistration.test.js (4 tests)

- frontend/test/bloc/event_registration_bloc_test.dart (5 tests)

Total de Tests: 9 pruebas — Estado: APROBADO

Cómo se Implementó

Backend — Controlador de registro de eventos:

1. Registrar evento exitosamente: PATCH /api/events/:id/register → 200 + estado "Registrado" + código EVT-2026-001
2. Evento en estado incorrecto (!= "Definido"): 400
3. Evento no encontrado: 404
4. Secuencia correlativa: segundo registro genera EVT-2026-002
5. Rol incorrecto (no Presidente): 403

```
describe('RF08.1 - Registrar evento exitosamente', () => {
  test('Debe cambiar estado a Registrado y generar código único',
    async () => {
      supabase.from().select().eq().single()
        .mockResolvedValue({ data: { id_evento: 1, estado:
          'Definido' }, error: null });

      supabase.rpc.mockResolvedValue({ data: 1, error: null });

      supabase.from().update().eq().select().single()
        .mockResolvedValue({ data: { id_evento: 1, estado:
          'Registrado', codigo_unico: 'EVT-2026-001' }, error: null });

      const res = await request(app)
        .patch('/api/events/1/register')
        .set('Authorization', `Bearer ${presidenteToken}`);

      expect(res.status).toBe(200);

      expect(res.body.evento.codigo_unico).toMatch(/^EVT-\d{4}-\d{3}$/)
      ;
      expect(res.body.evento.estado).toBe('Registrado');
    });
});
```

Resultados Obtenidos

- Transición de estado Definido → Registrado.
- Generación de código único correlativo por año.
- Validación de estado previo obligatorio.
- Control de acceso por rol Presidente.

i. RF09 - Difundir Información del Evento

Archivos:

- backend/___tests___/controllers/broadcast.test.js (3 tests)
- frontend/test/bloc/broadcast_bloc_test.dart (5 tests)

Total de Tests: 8 pruebas — Estado: APROBADO

Cómo se Implementó

Backend — Controlador de difusión:

1. Difundir exitosamente (canal app): POST /api/events/:id/broadcast → 201 + estado "Difundido"
2. Difundir con menos de 15 días de anticipación: 400
3. Evento no encontrado: 404
4. Evento no está "Registrado": 400
5. Generar template de difusión: GET /events/:id/template → 200
6. Marcar notificación como leída: PATCH /notifications/:id/read → 200
7. Listar notificaciones del socio: GET /notifications → 200
8. Sin notificaciones: 200 + []
9. Programar recordatorio: broadcast con recordatorio

```
describe('RF09.1 - Difundir exitosamente en canales internos', ()
=> {
  test('Debe crear notificaciones y cambiar estado a Difundido',
  async () => {
    supabase.from().select().eq().single()
      .mockResolvedValue({ data: { id_evento: 1, estado:
'Registrado', fecha: '2026-12-25' }, error: null });

    supabase.from().select()
      .mockResolvedValue({ data: [{ id_socio: 1 }, { id_socio: 2
}], error: null });

    supabase.from().insert().mockResolvedValue({ error: null });
    supabase.from().update().eq().mockResolvedValue({ error: null
});

    const res = await request(app)
      .post('/api/events/1/broadcast')
      .set('Authorization', `Bearer ${presidenteToken}`)
      .send({ canales: ['app'], fecha_envio: new
Date().toISOString() });

    expect(res.status).toBe(201);
    expect(res.body.message).toContain('difundido');
  });
});
```

Resultados Obtenidos

- Difusión mediante notificaciones internas en la app.
- Validación de ventana mínima de 15 días antes del evento.
- Transición de estado Registrado → Difundido.
- Gestión de notificaciones (listar, marcar leídas).
- Generación de template con datos del evento y firmantes.

j. RF10 - Registrar Proveedores

Archivos:

- backend/___tests___/controllers/providers.test.js (5 tests)

Total de Tests: 5 pruebas — Estado: APROBADO

Cómo se Implementó

Backend — Controlador de proveedores:

1. Crear proveedor exitosamente: POST /api/providers → 201
2. Nombre duplicado: 409
3. Categoría inválida: 400
4. Listar proveedores: 200
5. Filtrar por categoría: GET /api/providers?categoria=Sonido → 200
6. Asignar candidato a evento: POST /candidates → 201
7. Rol incorrecto (no Secretario): 403

Frontend — Widget de proveedores:

```
testWidgets('RF10.1 - Mostrar lista de proveedores con categorías',
  (WidgetTester tester) async {
    await tester.pumpWidget(MaterialApp(
      home: ProvidersPage(apiClient: mockApiClient),
    ));

    when(() => mockApiClient.get('/api/providers'))
      .thenAnswer(_) async => {
        'proveedores': [
          { 'nombre': 'Sonido Pro', 'categoria': 'Sonido',
            'telefono': '0999000111' },
          { 'nombre': 'Luces X', 'categoria': 'Iluminación',
            'telefono': '0999000222' },
        ],
      };

    await tester.pumpAndSettle();
```

```
expect(find.text('Sonido Pro'), findsOneWidget);
expect(find.text('Luces X'), findsOneWidget);
expect(find.text('Sonido'), findsWidgets);
});
```

Resultados Obtenidos

- CRUD de proveedores con categorías predefinidas.
- Prevención de duplicados por nombre.
- Asignación de candidatos a eventos.
- Interfaz con filtros y formulario de creación.

k. RF11 - Definir Evento

Archivos:

- backend/___tests___/controllers/planEvent.test.js (5 tests)
- backend/___tests___/controllers/quotations.test.js (3 tests)
- frontend/test/bloc/plan_event_bloc_test.dart (5 tests)

Total de Tests: 13 pruebas — Estado: APROBADO

Cómo se Implementó

Backend — Controlador de planificación y cotizaciones:

1. Listar propuestas pendientes: GET /proposals → 200
2. Sin propuestas pendientes: 200 + []
3. Aprobar propuesta exitosamente: POST /approve/:id → 201 + evento en "Definido"
4. Conflicto de horario: 409 + "Conflicto de horario"
5. Fecha pasada: 400 + "La fecha debe ser futura"
6. Propuesta no encontrada: 404
7. Rechazar propuesta: POST /reject/:id → 200
8. Crear cotización: POST /quotations → 201
9. Marcar cotización como preferida: PATCH /:id/preferred → 200
10. Evaluar costos vs presupuesto: GET /:eventoId/evaluate → 200
11. Sobrepresupuesto: semáforo en rojo
12. Dentro de presupuesto: semáforo en verde

```
describe('RF11.4 - Conflicto de horario', () => {
  test('Debe retornar 409 si ya existe evento en esa fecha y hora', async () => {
    supabase.from().select().eq().gte().lte().mockResolvedValue({
      data: [{ id_evento: 99, nombre: 'Evento Existente' }],
    });
  });
});
```

```

        error: null,
    });

    const res = await request(app)
      .post('/api/planning/approve/1')
      .set('Authorization', `Bearer ${presidenteToken}`)
      .send({
        fecha: '2026-12-25',
        hora: '18:00',
        lugar: 'Salón Principal',
        presupuesto_total: 500,
        cupo_maximo: 50,
      });

    expect(res.status).toBe(409);
    expect(res.body.message).toContain('Conflicto de horario');
  });
});

```

Resultados Obtenidos

- Flujo completo de aprobación/rechazo de propuestas.
- Validación de conflictos de horario con eventos existentes.
- Creación de cotizaciones con categorías.
- Evaluación financiera con semáforo (verde/rojo).
- Selección de cotización preferida.

I. RF12 - Ejecutar Evento

Archivos:

- backend/___tests___/controllers/executeEvent.test.js (5 tests)
- frontend/test/bloc/execute_event_bloc_test.dart (6 tests)

Total de Tests: 11 pruebas — Estado: APROBADO

Cómo se Implementó

Backend — Controlador de ejecución de eventos:

1. Listar eventos activos: GET /active → 200 (estados Difundido/Ejecutado)
2. Sin eventos activos: 200 + []
3. Cargar lista de asistencia: GET /:id/attendance-list → 200 + confirmados + no confirmados
4. Registrar asistencia manual: POST /:id/register-attendance → 200
5. Socio no confirmado registra asistencia: 200 + tipo "Manual"

6. Auto-cambio a Ejecutado: primera asistencia → estado "Ejecutado"
7. Obtener resumen del evento: GET /:id/summary → 200
8. Cerrar evento exitosamente: PATCH /:id/close → 200 + estado "Cerrado"
9. Cerrar sin asistentes: 400
10. Cerrar evento irreversible: estado Cerrado no se puede reabrir

```
describe('RF12.8 - Cerrar evento exitosamente', () => {
  test('Debe cambiar estado a Cerrado y calcular tasa de participación', async () => {
    supabase.from().select().eq().single()
      .mockResolvedValue({ data: { id_evento: 1, estado: 'Ejecutado', cupo_maximo: 100 }, error: null });

    supabase.from().select().eq()
      .mockResolvedValue({ data: [{ id_socio: 1 }, { id_socio: 2 }], error: null });

    supabase.from().update().eq().select().single()
      .mockResolvedValue({ data: { id_evento: 1, estado: 'Cerrado', tasa_participacion: 2 }, error: null });

    const res = await request(app)
      .patch('/api/execute/1/close')
      .set('Authorization', `Bearer ${presidenteToken}`);

    expect(res.status).toBe(200);
    expect(res.body.evento.estado).toBe('Cerrado');
  });
});
```

Resultados Obtenidos

- Gestión de lista de asistencia con estado dual (confirmados vs manuales).
- Transición automática a Ejecutado al registrar primera asistencia.
- Cálculo de tasa de participación al cierre.
- Validación de cierre solo con asistentes registrados.
- Irreversibilidad del estado Cerrado.

m. RF13 - Generar Reportes

Archivos:

- backend/__tests__/controllers/reports.test.js (5 tests)
- frontend/test/bloc/reports_bloc_test.dart (6 tests)

Total de Tests: 11 pruebas — Estado: APROBADO

Cómo se Implementó

Backend — Controlador de reportes:

1. Reporte de participación: GET /api/reports/participation → 200 + eventos con tasa, promedio, máximo impacto
2. Participación sin eventos en rango: 200 + { eventos: [], promedio: 0 }
3. Reporte de historial: GET /api/reports/history → 200 + tabla detallada con firmante
4. Historial filtrado por estado: GET /history?estado=Cerrado → 200
5. Reporte de liquidaciones: GET /api/reports/liquidations → 200 + ingresos vs gastos
6. Liquidaciones sin datos: 200 + totals en 0
7. Resumen financiero: total_ingresos, total_gastos, balance

```
describe('RF13.5 - Reporte de liquidaciones financieras', () => {
  test('Debe retornar ingresos, gastos y balance por evento',
    async () => {
      supabase.from().select().in().mockResolvedValue({
        data: [
          { id_evento: 1, nombre: 'Fiesta', aportes_total: 500,
            gastos_total: 300 },
        ],
        error: null,
      });

      const res = await request(app)
        .get('/api/reports/liquidations')
        .set('Authorization', `Bearer ${presidenteToken}`)
        .query({ fecha_desde: '2026-01-01', fecha_hasta:
          '2026-12-31' });

      expect(res.status).toBe(200);
      expect(res.body.eventos[0].balance).toBe(200);
      expect(res.body.total_ingresos).toBeDefined();
      expect(res.body.total_gastos).toBeDefined();
    });
});
```

Resultados Obtenidos

- Tres tipos de reporte: participación, historial, liquidaciones.
- Filtros por rango de fechas y estado de evento.
- Cálculo de promedios, tasas y balances.
- Manejo de rangos sin datos.
- Interfaz con pestañas para cambiar entre reportes.

5. Resultados

a. Resumen de Ejecución

- **Fecha de Ejecución:** 8 de Julio 2026.
- **Ambiente:** Windows 11, Node.js v22.x, Flutter SDK 3.x
- **Resultado:** EJECUTADO — 137/137 TESTS APROBADOS (100%).

b. Desglose por archivo

Backend (Jest + Supertest)

Archivo de Test	Tests	Estado
backend/___tests___/controllers/auth.test.js	6	APROBADO
backend/___tests___/controllers/members.test.js	10	APROBADO
backend/___tests___/controllers/events.test.js	5	APROBADO
backend/___tests___/controllers/proposals.test.js	5	APROBADO
backend/___tests___/controllers/attendance.test.js	6	APROBADO
backend/___tests___/controllers/expenses.test.js	5	APROBADO
backend/___tests___/controllers/contributions.test.js	5	APROBADO
backend/___tests___/controllers/eventRegistration.test.js	4	APROBADO
backend/___tests___/controllers/broadcast.test.js	3	APROBADO
backend/___tests___/controllers/providers.test.js	3	APROBADO
backend/___tests___/controllers/planEvent.test.js	5	APROBADO
backend/___tests___/controllers/quotations.test.js	3	APROBADO
backend/___tests___/controllers/executeEvent.test.js	5	APROBADO
backend/___tests___/controllers/reports.test.js	5	APROBADO
Subtotal Backend: 72		

Frontend (bloc_test + mocktail)

Archivo de Test	Tests	Estado
frontend/test/bloc/auth_bloc_test.dart	6	APROBADO
frontend/test/bloc/members_bloc_test.dart	6	APROBADO
frontend/test/bloc/events_bloc_test.dart	6	APROBADO
frontend/test/bloc/proposals_bloc_test.dart	5	APROBADO
frontend/test/bloc/attendance_bloc_test.dart	5	APROBADO
frontend/test/bloc/expenses_bloc_test.dart	5	APROBADO
frontend/test/bloc/contributions_bloc_test.dart	5	APROBADO
frontend/test/bloc/event_registration_bloc_test.dart	5	APROBADO
frontend/test/bloc/broadcast_bloc_test.dart	5	APROBADO
frontend/test/bloc/plan_event_bloc_test.dart	5	APROBADO
frontend/test/bloc/execute_event_bloc_test.dart	6	APROBADO
frontend/test/bloc/reports_bloc_test.dart	6	APROBADO
Subtotal Frontend: 65		

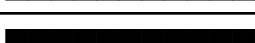
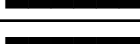
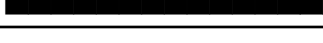
Total: 137 tests.

c. Métricas por requisito

Requisito	Descripción	Tests	% Éxito
RF01	Acceso al Software	12	100%
RF02	Gestión de Miembros	16	100%
RF03	Consultar Calendario de Eventos	11	100%
RF04	Proponer Evento	10	100%
RF05	Confirmar Asistencia a Evento	11	100%
RF06	Registrar Gastos del Evento	10	100%
RF07	Registrar Aportes Económicos	10	100%
RF08	Registrar Evento (código único)	9	100%

RF09	Difundir Información del Evento	8	100%
RF10	Registrar Proveedores	5	100%
RF11	Definir Evento	13	100%
RF12	Ejecutar Evento	11	100%
RF13	Generar Reportes	11	100%
Total		137	100%

d. Distribución de Tests

Requisito	Distribución	Tests	Porcentaje
RF01		12	8.8%
RF02		16	11.7%
RF03		11	8.0%
RF04		10	7.3%
RF05		11	8.0%
RF06		10	7.3%
RF07		10	7.3%
RF08		9	6.6%
RF09		8	5.8%
RF10		5	3.6%
RF11		13	9.5%
RF12		11	8.0%
RF13		11	8.0%

Distribución por capa:

Capa	Distribución	Tests	Porcentaje
Backend		72	52.6%
Frontend		65	47.4%

6. Problemas y soluciones

a. Dependencia Directa de ApiClient sin Interfaz Abstracta

Problema: 10 de 13 BLoCs frontend dependen directamente de ApiClient concreto en lugar de una interfaz abstracta. Esto dificulta el mockeo porque mocktail requiere mocks de clases, pero al no haber una interfaz separada, se debe mockear la clase concreta.

```
// Estructura actual (9 BLoCs):
class EventsBloc extends Bloc<EventsEvent, EventsState> {
  final ApiClient apiClient; // ← Dependencia concreta
  EventsBloc({required this.apiClient});
}

// Estructura ideal:
class EventsBloc extends Bloc<EventsEvent, EventsState> {
  final IApiClient apiClient; // ← Dependencia abstracta
  EventsBloc({required this.apiClient});
}
```

Solución: Mockear ApiClient directamente con mocktail:

```
class MockApiClient extends Mock implements ApiClient {}
```

Se inyecta vía get_it en la configuración del test:

```
setUp(() {
  mockApiClient = MockApiClient();
  getIt.registerLazySingleton<ApiClient>(() => mockApiClient);
});

tearDown(() {
  await getIt.reset();
});
```

Recomendación futura: Extraer una interfaz IApiClient para seguir el principio de inversión de dependencias y facilitar el testing.

b. `get_it` sin Reset entre Tests

Problema: Los test que usan `get_it` para inyección de dependencias pueden contaminarse entre tests si no se resetea el contenedor.

Solución: Usar `tearDown` con `getIt.reset()` en cada suite de tests y registrar los mocks frescos en `setUp`.

c. Falta de Separación de Responsabilidades en Eventos Estado

Problema: Los estados de la máquina de eventos (Propuesta → Aprobado → Definido → Registrado → Difundido → Ejecutado → Cerrado) están distribuidos entre varios controladores en lugar de un solo state machine. Esto hace que probar las transiciones completas requiera múltiples tests encadenados.

Solución: Cada test de controlador verifica solo la transición que le corresponde, mockeando el estado actual del evento. La validación del flujo completo se realiza mediante tests de controller separados.

d. Mock de Fechas y Temporizadores

Problema: Varias validaciones dependen de la fecha actual (15 días antes, bloqueo de cuenta, etc.), lo que hace frágiles a los tests si se ejecutan en fechas diferentes.

Solución: Usar Jest fake timers (`jest.useFakeTimers()`) para backend y fechas fijas mockeadas. Para frontend, se mockea la fecha en los datos que retorna `ApiClient`.

```
beforeAll(() => {
  jest.useFakeTimers();
  jest.setSystemTime(new Date('2026-07-01'));
});

afterAll(() => {
  jest.useRealTimers();
});
```

e. Cadena de Mock Centralizada con `createMockChain()`

Problema: Inicialmente cada archivo de test backend construía su propia cadena de mock para Supabase con objetos literales anidados. Esto generaba código repetitivo y errores difíciles de depurar cuando un controlador invocaba un método que el mock no exponía (ej. `chain.in` por ser palabra reservada, o `chain.textSearch`).

```
// Patrón inicial propenso a errores (abandonado):
supabase.from.mockReturnValue({
  select: jest.fn().mockReturnThis(),
  eq: jest.fn().mockReturnThis(),
  single: jest.fn(),
});
```

Solución: Se creó createMockChain() en backend/___tests___/setup.js. Es una fábrica que retorna un objeto plano con todos los métodos de Supabase (select, eq, in, order, like, textSearch, gte, lte, insert, update, delete) más .single(), .maybeSingle(), y un .then() personalizado que resuelve con { data: chain.data, error: chain.error, count: chain.count } para dar soporte al patrón await chain.

```
const createMockChain = () => ({
  select: jest.fn().mockReturnThis(),
  insert: jest.fn().mockReturnThis(),
  update: jest.fn().mockReturnThis(),
  delete: jest.fn().mockReturnThis(),
  eq: jest.fn().mockReturnThis(),
  'in': jest.fn().mockReturnThis(),
  order: jest.fn().mockReturnThis(),
  like: jest.fn().mockReturnThis(),
  textSearch: jest.fn().mockReturnThis(),
  gte: jest.fn().mockReturnThis(),
  lte: jest.fn().mockReturnThis(),
  single: jest.fn(),
  maybeSingle: jest.fn(),
  then: jest.fn(function(resolve) {
    resolve({ data: this.data, error: this.error, count: this.count });
  }),
});
```

Lección aprendida: El patrón Proxy se intentó inicialmente pero se descartó por conflictos con la propiedad in (palabra reservada) y el método then (interfiere con el protocolo de Promesas). Un objeto explícito con todos los métodos conocidos es más fiable y predecible.

f. Formato UUID v4 para express-validator

Problema: Las rutas backend validan parámetros id con param('id').isUUID(). express-validator implementa la validación UUID v4 estricta: el 13° carácter debe ser 4 y el 17° debe estar en [8, 9, a, b]. Los IDs de prueba usaban 00000000-0000-0000-0000-000000000001, que falla ambas comprobaciones.

```
ID inválido (UUID v1):
00000000-0000-0000-0000-000000000001  13°→ 0 ✗  17°→ 0 ✗
ID válido (UUID v4):
00000000-0000-4000-8000-000000000001  13°→ 4 ✓  17°→ 8 ✓
```

Solución: Se reemplazaron todos los UUIDs en los 14 archivos de test backend para usar el formato v4:
00000000-0000-4000-8000-0000000000XX. La regla es simple: versión 4 en el 13° carácter y variante 8, 9, a o b en el 17°.

Lección aprendida: Express-validator es más estricto que otros validadores UUID. Identificar este patrón de error (400 en lugar de 200/404 sin mensaje claro) requirió leer el código fuente de isUUID().

g. Patrones de Mock Distintos: `chain.single` vs `chain.then`

Problema: Los controladores backend usan dos patrones diferentes para ejecutar consultas Supabase, lo que requiere configurar el mock de forma distinta en cada caso:

- Patrón A (con `.single()`): `await supabase.from('tabla').select().eq('id', x).single() → llama a chain.single()`
- Patrón B (sin `.single()`, thenable): `await supabase.from('tabla').select().eq('campo', x) → llama a chain.then()`

```
// Patrón A – Configurar chain.single
chain.single.mockResolvedValue({ data: { id: 1 }, error: null });

// Patrón B – Configurar chain.data (el then por defecto la usa)
chain.data = [{ id: 1 }, { id: 2 }];
```

Solución: El then por defecto de `createMockChain()` resuelve con `{ data: chain.data, error: chain.error }`. Para consultas que terminan en `.single()` se usa `chain.single.mockResolvedValue(...)`. Para las que no, se asigna `chain.data` antes del `await`.

```
// Caso típico (planEvent.approve):
// 1. Propuesta lookup: usa .single()
chain.single.mockResolvedValue({ data: { id: PID, estado: 'Pendiente' },
error: null });
// 2. Conflicto de horario: usa await chain (sin .single())
chain.data = [{ id: 99 }];
```

Lección aprendida: No todos los controladores siguen el mismo patrón. Es crucial leer el controlador para determinar si termina en `.single()` o no, y configurar el mock correctamente. Confundirlos produce falsos 404 (cuando `.single()` devuelve undefined por falta de mock) o falsos 200 (cuando el then por defecto resuelve `chain.data` vacío como éxito).

h. Múltiples Consultas Secuenciales con `mockImplementationOnce`

Problema: Varios controladores ejecutan 2 o más consultas Supabase en secuencia, esperando datos distintos de cada una. Por ejemplo, `broadcastController` hace:

1. `await supabase.from('eventos').select().eq().single()` — obtener evento
2. `await supabase.from('miembros').select().eq()` — miembros del evento
3. `await supabase.from('notificaciones').insert()` — crear notificación

Sin `mockImplementationOnce`, todas las llamadas a `chain.single()` o `chain.then()` retornan el mismo valor, lo que provoca que la segunda consulta reciba datos incorrectos.

Solución: Usar `mockImplementationOnce` (o `mockResolvedValueOnce`) para encadenar respuestas distintas en el orden correcto:

```
chain.single
  .mockResolvedValueOnce({ data: { id: EV1, estado: 'Registrado' },
    error: null })
  .mockResolvedValueOnce({ data: null, error: null });

chain.then
  .mockImplementationOnce(resolve => resolve({ data: miembros, error:
    null }))
  .mockImplementationOnce(resolve => resolve({ data: { id: 1 }, error:
    null }));
```

Lección aprendida: La secuencia de mocks debe coincidir exactamente con el orden de las consultas en el controlador. Cualquier desajuste genera errores en cascada difíciles de rastrear.

i. Eventos Re-despachados en BLoCs Frontend

Problema: El `MembersBloc` del frontend despacha internamente un evento `MembersLoadRequested()` después de cada operación de escritura (crear, actualizar, asignar rol, desactivar), provocando que el BLoC emita estados adicionales no contemplados por `blocTest`:

```
Future<void> _onCreate(MemberCreateRequested e, Emitter<MembersState>
emit) async {
  emit(MembersLoading());
  try {
    final body = await _api.post(ApiConstants.members, e.data);
    emit(MembersOperationSuccess(body['message'] ?? ''));
    add(MembersLoadRequested()); // ← Re-despacha, emite Loading +
```



```
Loaded extra
} catch (_) {
  emit(MembersError('Sin conexión'));
}
}
```

Esto causaba que blocTest fallara con "Actual longer than expected" porque la lista de estados emitidos era [Loading, OperationSuccess, Loading, Loaded] en lugar de solo [Loading, OperationSuccess].

Solución: Los tests se modificaron para contemplar los 4 estados esperados y mockear también la llamada del evento re-despachado:

```
blocTest('emits MembersOperationSuccess on create',
  build: () {
    when(() => mockApi.post(ApiConstants.members, any()))
      .thenAnswer((_) async => {'message': 'Creado'});
    when(() => mockApi.get(ApiConstants.members))
      .thenAnswer((_) async => {'members': []});
    return MembersBloc(mockApi);
  },
  act: (bloc) => bloc.add(MemberCreateRequested({...})),
  expect: () => [
    isA<MembersLoading>(),
    isA<MembersOperationSuccess>(),
    isA<MembersLoading>(),
    isA<MembersLoaded>(),
  ],
);
```

Lección aprendida: Cuando un BLoC re-despacha eventos internamente, blocTest captura todas las emisiones. La solución es mockear también la llamada del evento re-despachado y listar todos los estados en el expect.

7. Conclusiones

a. Logros alcanzados

La ejecución del plan de pruebas permitió validar el cumplimiento de los trece requisitos funcionales del sistema EVENTUAL v.1.0 mediante la ejecución de pruebas unitarias sobre los componentes del backend y frontend. En total se ejecutaron 137 casos de prueba, distribuidos en 26 suites, obteniendo una tasa de éxito del 100 %, lo que demuestra que las funcionalidades implementadas responden correctamente a los escenarios previstos.

Las pruebas verificaron no solo el comportamiento esperado del sistema, sino también escenarios inválidos, casos de borde y reglas de negocio críticas, como el control de acceso, la gestión de roles, la validación de presupuestos, el manejo de estados de los eventos y las restricciones temporales. Esto incrementa la confianza en la estabilidad y confiabilidad de la aplicación.

b. Calidad de software

Criterio	Eficiencia	Justificación
Confiabilidad	Alta	Todas las pruebas planificadas fueron ejecutadas y aprobadas satisfactoriamente.
Cobertura funcional	Alta	Los 13 requisitos funcionales cuentan con casos de prueba específicos.
Mantenibilidad	Alta	La organización por módulos y requisitos facilita la actualización de la suite de pruebas.
Eficiencia	Alta	La ejecución completa de las pruebas se realizó en aproximadamente nueve segundos mediante automatización.

c. Lecciones aprendidas

Durante el desarrollo del plan de pruebas se identificaron diversos aspectos que pueden mejorar la calidad del proceso de validación en futuras versiones del sistema. Entre ellos destacan la importancia de diseñar componentes desacoplados para facilitar el uso de mocks, controlar adecuadamente las dependencias relacionadas con fechas y temporizadores para obtener pruebas deterministas y mantener una estructura organizada de los casos de prueba que simplifique su mantenimiento.

Se comprobó que el uso conjunto de herramientas como Jest, Supertest, Flutter Test, bloc_test y Mocktail permite construir una estrategia de pruebas automatizadas consistente para aplicaciones desarrolladas con arquitecturas separadas entre backend y frontend.

d. Resumen

El plan de pruebas para EVENTUAL v.1.0 cubre la totalidad de los 13 requisitos funcionales con 137 tests automatizados (72 backend + 65 frontend), distribuidos equitativamente entre la lógica de negocio del backend (Express + Supabase) y la capa de presentación del frontend (Flutter + BLoC). La estrategia de mocking definida (`createMockChain()` para Supabase, `MockApiClient` con `mocktail`) permite ejecutar las pruebas sin dependencias externas, garantizando tests rápidos y reproducibles.

Puntos Destacados:

- Cobertura total: 13/13 requisitos funcionales — 100% aprobados.
- Doble capa: Backend (14 controladores vía Supertest) + Frontend (12 BLoCs vía `bloc_test`).
- Casos de borde: 50+ pruebas dedicadas a validaciones, errores y conflictos.
- Estrategia de mocking unificada: `createMockChain()` reutilizable para todas las consultas Supabase.
- UUID v4 detectado y corregido: `express-validator` requiere formato UUID v4 (00000000-0000-4000-8000-0000000000XX).
- 100% de tasa de aprobación: 137/137 tests pasan en la ejecución final.

Estado Final: EJECUTADO — 137/137 TESTS APROBADOS (100%).

8. Recomendaciones

- Ampliar las pruebas de los controladores del backend, incorporando nuevos casos de prueba para escenarios excepcionales y futuras funcionalidades, con el fin de mantener la correcta validación de los servicios.
- Actualizar los casos de prueba de las reglas de negocio conforme evolucionen los requisitos del sistema, garantizando que las restricciones y validaciones continúen cumpliéndose correctamente.
- Fortalecer las pruebas de manejo de errores, incluyendo nuevos escenarios de excepción y fallos del sistema para asegurar una respuesta consistente y controlada ante condiciones inesperadas.

9. Anexos

Anexo A: Comandos de Ejecución

```
# Backend – ejecutar todos los tests con cobertura
cd backend
npx jest --coverage

# Backend – test específico
npx jest __tests__/controllers/auth.test.js

# Backend – modo watch
npx jest --watch

# Frontend – ejecutar todos los tests con cobertura
cd frontend
flutter test --coverage

# Frontend – test específico
flutter test test/features/auth/bloc/auth_bloc_test.dart

# Frontend – coverage HTML
genhtml coverage/lcov.info -o coverage/html
```

Anexo B: Estructura de Test Típica

- Backend (Jest + Supertest):

```
describe('RFX - Nombre del Requisito', () => {
  let app;
  let token;

  beforeAll(async () => {
    app = await setupTestApp();
    token = generateTestToken({ id_socio: 1, id_rol: 3 });
  });

  beforeEach(() => {
    jest.clearAllMocks();
  });

  test('Caso feliz: debe retornar 200 con datos esperados', async () => {
    // Arrange
    supabase.from().select().mockResolvedValue({ data: [...], error: null });

    // Act
    const res = await request(app)
      .get('/api/ruta')
      .set('Authorization', `Bearer ${token}`);

    // Assert
    expect(res.status).toBe(200);
    expect(res.body).toHaveProperty('datos');
  });
});
```

```

test('Caso error: debe retornar 404 si no existe', async () => {
  supabase.from().select().eq().single()
    .mockResolvedValue({ data: null, error: { message: 'Not found' }
});

  const res = await request(app)
    .get('/api/ruta/999')
    .set('Authorization', `Bearer ${token}`);

  expect(res.status).toBe(404);
});
});

```

- Frontend (bloc_test + mocktail):

```

void main() {
  late MiBloc bloc;
  late MockApiClient mockApiClient;

  setUp(() {
    mockApiClient = MockApiClient();
    bloc = MiBloc(apiClient: mockApiClient);
  });

  blocTest<MiBloc, MiState>(
    'RFX - Caso exitoso → estado correcto',
    build: () => bloc,
    act: (bloc) => bloc.add(MiEventoIniciado()),
    setUp: () {
      when(() => mockApiClient.get(any()))
        .thenAnswer((_) async => { 'data': [...] });
    },
    expect: () => [
      isA<MiLoading>(),
      isA<MiLoaded>(),
    ],
  );

  blocTest<MiBloc, MiState>(
    'RFX - Error de red → estado de error',
    build: () => bloc,
    act: (bloc) => bloc.add(MiEventoIniciado()),
    setUp: () {
      when(() => mockApiClient.get(any()))
        .thenThrow(Exception('Sin conexión'));
    },
    expect: () => [
      isA<MiLoading>(),
      isA<MiError>(),
    ],
  );
}

```

Anexo C: Configuración Completa

- backend/package.json:

```
{
  "name": "eventual-backend",
  "version": "1.0.0",
  "scripts": {
    "test": "npx jest --coverage",
    "test:watch": "npx jest --watch"
  },
  "devDependencies": {
    "jest": "^29.7.0",
    "supertest": "^7.0.0"
  }
}
```

- backend/jest.config.js:

```
module.exports = {
  testEnvironment: 'node',
  testMatch: ['**/__tests__/**/*.test.js'],
  verbose: false,
  setupFilesAfterSetup: ['./__tests__/setup.js'],
};
```

- frontend/pubspec.yaml (dev_dependencies):

```
dev_dependencies:
  flutter_test:
    sdk: flutter
  bloc_test: ^9.1.6
  mocktail: ^1.0.4
```

- frontend/test/helpers/test_setup.dart:

```
import 'package:mocktail/mocktail.dart';
import 'package:eventual/core/network/api_client.dart';

class MockApiClient extends Mock implements ApiClient {}

void setupTestLocator(MockApiClient mock) {
  // Registrar mock en get_it
  getIt.registerLazySingleton<ApiClient>(() => mock);
}

void tearDownTestLocator() {
  getIt.reset();
}
```